MidCourse Progress Report

Developing a Diabetes and Prediabetes Prediction Web App Using 2025 CDC BRFSS Data

Course Name: Applied Research Project

Section #: CSIS-4495-002

Student Name: Liqin Shi

Student ID: 300431612

Project arrangement: Individual project

Submission date: October 26, 2026

Video Demonstration link: Video Demo Link

Submission Checklist

Status	Requirement	Current position / action
☒	GitHub Repository Code Check-In	Current notebook, saved model files, and Flask implementation have been checked in throughout development. GitHub Desktop history shows regular small commits for preprocessing, baseline models, class weighting, oversampling, model comparison, final model selection, CANRISK research, Flask setup, form submission, and connection of the web app to the final model.
☒	Video Demo of the Product	Completed a public video demonstration of the implemented work
☒	Video Link	Video Demo Link
☒	Installation Instructions	README updated with project description, dependencies, and installation instructions.
☒	Blackboard Submission	Submitted the final PDF on Blackboard.
☒	GitHub MidCourse Report Check-In	Checked the final MidCourse Progress Report PDF into DocumentsAndReports.

1. Introduction

Diabetes in Canada:

Diabetes is a chronic condition that develops when your body does not produce enough insulin or cannot effectively use insulin, which it needs to turn sugar into energy. It is one of the most common chronic diseases affecting people living in Canada. Around 3.9 million people in Canada over a year old live with diagnosed diabetes (9.7% of the population). This number excludes gestational diabetes. In addition, over 6% of adults in Canada live with prediabetes, which gives them a higher chance of developing type 2 diabetes [6].

Impact of diabetes

Diabetes complications are associated with premature death. Diabetes can reduce lifespan by **five to 15 years**. It is estimated that the all-cause mortality rate among people living with diabetes is twice as high as the all-cause mortality rate for those without diabetes. Diabetes contributes to[7]:

- 40% of heart attacks
- 30% of strokes
- 50% of kidney failure requiring dialysis
- Leading cause of blindness
- 70% of all non-traumatic leg and foot amputations

Prevention and risk factors

There's currently no cure for diabetes, but steps can be taken to control it and lower your risk. A large number of adults may not know that they have abnormal blood glucose. The purpose of this project is to develop a simple diabetes and prediabetes risk screening tool using the 2025 BRFSS data. The Behavioral Risk Factor Surveillance System (BRFSS) is a large public health survey conducted by the U.S. Centers for Disease Control and Prevention (CDC) **[1]**. Users can enter a short set of self-reported health, lifestyle, and demographic information and receive a model-based classification of No Diabetes, Prediabetes, or Diabetes.

The goal is not to diagnose diabetes. Instead, the tool is designed to support early self-awareness. If a user receives a higher-risk result, the application can encourage the user to seek professional medical advice and appropriate blood testing. The project also examines common risk-related factors, such as BMI, physical activity, blood pressure, smoking, and general health, so that users can better understand which lifestyle and health factors may be associated with diabetes and prediabetes.

The technical goal is to build an end-to-end data product that includes source documentation, data cleaning, model development and evaluation, saved model files, and a working Flask web application.

1.1 Research questions

1. Which BRFSS fields are practical and defensible inputs for a short web form?
2. Which modeling approach gives the most useful balance across No Diabetes, Diabetes, and especially the rare Prediabetes class?
3. Can a saved preprocessing pipeline and model accept new web-form input and return a clear three-class result?
4. Do a small number of hypothetical scenario tests show a broadly reasonable direction when compared with established public risk-screening tools?

1.2 Relevant existing work and research gap

BRFSS is a large population health survey covering chronic conditions, health behaviours, access to care, and demographics[1]. It was not designed as a diabetes-only dataset. This makes it useful for testing a practical prediction pipeline, but it also creates limitations: the diabetes target is based on a respondent reporting that a health professional told them about diabetes or prediabetes[2], not on a common A1C or glucose test administered to every participant.

Two public tools are useful reference points. The CDC Prediabetes Risk Test uses a fixed point score based on a small set of known risk factors, and a score of 5 or more is considered high risk[3][4]. Canada's CANRISK questionnaire also uses a fixed risk-score approach and includes factors such as age, BMI, waist circumference, activity, high blood pressure, prior high blood sugar, family history, ethnicity, and education. My project is different: it learns coefficients from the 2025 BRFSS data and returns scores for three reported-status classes. Therefore, these tools are comparison references, not ground truth for validating my model.

1.3 Initial assumptions and expected benefit

I expected age, BMI, high blood pressure, physical activity, and general health to be useful. I also expected Prediabetes to be the hardest class because it is less common and can overlap with both No Diabetes and Diabetes. The expected benefit is not a clinical diagnostic tool. The benefit is a reproducible course project that demonstrates how a large public-health dataset can be turned into a working machine-learning application while documenting limitations honestly.

2. Summary of the Initially Proposed Research Project

The proposal defined an individual project using the 2025 CDC BRFSS combined landline and cellphone public-use dataset. The original working file contained 356,158 rows and 283 columns. DIABETE4 was selected as the source for a three-class target: No Diabetes, Prediabetes/Borderline Diabetes, and Diabetes. Direct leakage variables that reveal the target were not to be used as predictors.

2.1 Originally proposed method

- Review the CDC codebook and calculated-variable documentation.
- Define the three-class target and choose a short list of user-answerable predictors.
- Clean and recode the selected variables; review missing values rather than applying one global rule.
- Create an 80/20 train/test split and preprocessing pipeline.
- Train Logistic Regression and Random Forest as the two main candidate models.
- Evaluate test performance using confusion matrices and class-level F1, with extra attention to Prediabetes.
- Save the selected model and connect it to a Flask web form.
- Create a few Power BI visuals, document installation, test the product, and prepare the final report/demo.

2.2 Originally proposed technology stack

Layer	Technology / purpose
Development	Windows 11; Jupyter Notebook; VS Code
Data / modeling	Python, pandas, scikit-learn
Visualization	Power BI
Backend	Flask
Frontend	HTML/CSS
Saved model	joblib .pkl files
Version control	GitHub / GitHub Desktop
Optional data storage	SQLite summary table

3. Changes to the Proposal and Reasons

The core research question and dataset have not changed, but the modeling approach and schedule changed after I saw the actual class distribution and model results. These changes were driven by evidence from the data rather than by adding complexity for its own sake.

Change	Reason / decision
Expanded model comparison beyond two baseline models	The proposal planned Logistic Regression and Random Forest. Baseline accuracy was high mainly because 82.42% of records were No Diabetes, while Prediabetes represented only 2.88%. Baseline Logistic Regression correctly identified no Prediabetes cases and baseline Random Forest identified only 1 of 2,033. I therefore tested class weighting, random oversampling, custom weights, and HistGradientBoosting.
Added a healthcare-access Model B	Because the target depends on whether a person reports being told by a health professional, I tested whether checkup, personal-doctor, and insurance variables improved Prediabetes classification. Model B produced almost the same Prediabetes F1 as Model A, so I returned to the simpler Model A.
Final model selected earlier than the original timeline	The proposal placed model training in late October/early November. Modeling was substantially completed by October 19. This allowed me to begin Flask implementation earlier.
Web app simplified to the core prediction workflow first	Instead of starting with styling, SQLite, and a more polished interface, I first built the minimum working path: 13 inputs → Flask POST → pandas DataFrame → saved preprocessor → saved model → predicted class and three scores. This reduced debugging risk and created a working product for the MidCourse demo.
Added external scenario comparison	I added two hypothetical cases and compared the direction of results with the CDC Prediabetes Risk Test and Canada CANRISK[5]. This was not part of the original proposal. It is only a reasonableness check because the tools use different predictors, targets, and scoring methods.
Power BI and SQLite moved behind the core app	Power BI remains planned for the second half. SQLite is not required for the current prediction path, so it is secondary and will only be kept if it adds a clear function rather than being included only because it appeared in the proposal.

4. Project Planning and Timeline: Current Period to End of Term

The project is ahead of the original modeling schedule but still has important product, visualization, testing, and documentation work remaining. As an individual project, I am responsible for all items below.

Period	Main work	Milestone / evidence	Status
Oct 25–31	Finish MidCourse report, README installation instructions, video demo, and repository check-in.	Working Flask demo; MidCourse PDF; public video link; README.	In progress
Nov 1–8	Build Power BI visuals and verify counts against notebook.	Small dashboard / selected descriptive visuals.	Planned
Nov 9–15	Improve web validation and result wording; add clear non-diagnostic notice; test plausible inputs.	Stable local app and test notes.	Planned
Nov 16–22	Continue external comparison and cross-checking of model results; decide whether SQLite adds useful functionality.	Final implementation scope and interpretation notes.	Planned
Nov 23–Dec 6	Final testing, repository cleanup, references, final report, presentation preparation.	README, final report, demo scenarios, presentation.	Planned
Dec 7–9	Submit final deliverables and present project.	Final submission and presentation.	Planned

4.1 Gantt-style project management chart

Work package	Oct 25–31	Nov 1–8	Nov 9–15	Nov 16–22	Nov 23–Dec 6	Dec 7–9
MidCourse report / README / video	●					
Power BI		●	●			
Flask testing / refinement	●	●	●	●		
External comparison / scope review			●	●	●	
Final report / repository cleanup				●	●	
Presentation					●	●

5. Implemented Features and Current Results

5.1 Data selection, cleaning, and exploratory analysis

I reviewed the 2025 BRFSS codebook and calculated-variable documentation before recoding the model inputs. The final Model A uses 13 predictors: Age, Sex, BMI, HighBP, Smoking_Status, HighChol, Physical_Activity, Education, Income_Group, Race_Ethnicity, Fruit_Per_Day, Vegetable_Per_Day, and General_Health. I used variable-specific rules for special responses such as "Don't know," "Refused," and missing values rather than treating all missing data in the same way. Only 59.72% of records were complete across all selected variables. Complete-case deletion would therefore remove about 40% of the data, so I did not use that approach. I removed rows with missing target status and kept predictor missingness for the preprocessing pipeline. The resulting modeling dataset contained 352,705 records.

Target class	Count	Share
No Diabetes	290,712	82.42%
Diabetes	51,827	14.69%
Prediabetes	10,166	2.88%

The imbalance became the main modeling issue. It also changed how I interpreted "good performance": accuracy by itself was not sufficient because a model could obtain a high overall accuracy while failing almost completely on Prediabetes.

5.2 Preprocessing and train/test preparation

I used an 80/20 stratified train/test split with random_state=42. The training set contains 282,164 rows and the test set contains 70,541 rows. Numeric features use median imputation and StandardScaler. Categorical features use most-frequent imputation and OneHotEncoder(handle_unknown="ignore"). The preprocessor is fitted on training data only and then applied to test data to avoid leakage. The 13 original predictors expand to 40 processed features, and the processed train/test data contain no missing values.

5.3 Model development and comparison

I started with the two models proposed in the proposal, then added imbalance-handling and comparison models after seeing the baseline results. The table below summarizes the main test-set results used for decisions.

Model	Accuracy	Diabetes F1	Prediabetes Recall	Prediabetes F1	Macro F1
Baseline Logistic Regression	.827	.227	.000	.000	.377
Weighted Logistic Regression	.625	.411	.317	.096	.423
Oversampled Logistic Regression	.625	.410	.319	.096	.423
Custom-weight Logistic Regression	.769	.439	.051	.064	.456
Baseline Random Forest	.821	.264	.000	.001	.388
Weighted Random Forest	.821	.230	.000	.001	.377
Oversampled Random Forest	.803	.347	.007	.012	.416
HistGradientBoosting	.614	.421	.293	.089	.420

Key interpretation: The custom-weight Logistic Regression produced the highest macro F1 in this comparison, but it reduced Prediabetes recall to 5.1%. Because detecting the small Prediabetes class is a central project goal, I did not select the model only on macro F1.

5.4 Final model selection

I selected Model A with Weighted Logistic Regression. Its overall accuracy is lower than the baseline models, but it identifies substantially more Diabetes and Prediabetes cases. Prediabetes recall is 31.7% and Prediabetes F1 is 0.096. The confusion matrix shows that 645 of 2,033 Prediabetes test cases were correctly classified; 848 were classified as Diabetes and 540 as No Diabetes.

Oversampled Logistic Regression was almost identical: Prediabetes recall 31.9%, the same Prediabetes F1, the same overall accuracy, and the same macro F1. Weighted Logistic Regression is simpler because it keeps the original training rows rather than expanding the training set to about 697,710 rows through random oversampling. Model B with three healthcare-access variables also gave essentially the same Prediabetes F1. I therefore chose the simpler Model A pipeline.

The poor Prediabetes precision remains important. Many records predicted as Prediabetes are not labeled Prediabetes in BRFSS. This is one reason the web output is presented as a model result rather than a medical diagnosis or true clinical probability.

6. Flask Web Application Implementation

The MidCourse product now includes a working local Flask application. The model preprocessing object and Weighted Logistic Regression model were saved from Jupyter with joblib as preprocessor.pkl and weighted_logistic_model.pkl. Flask loads these files when the application starts, so the web app does not retrain the model for each user.

6.1 Current prediction workflow

1. User completes the 13-field HTML form.

2. The browser submits the form to Flask using POST.

3. Flask reads the values and creates a one-row pandas DataFrame with the same feature names used during training.

4. The saved preprocessor transforms the new row.

5. The saved Weighted Logistic Regression model returns a predicted class and three class scores.

6. The page displays the predicted class and converts the three scores to percentages for readability.

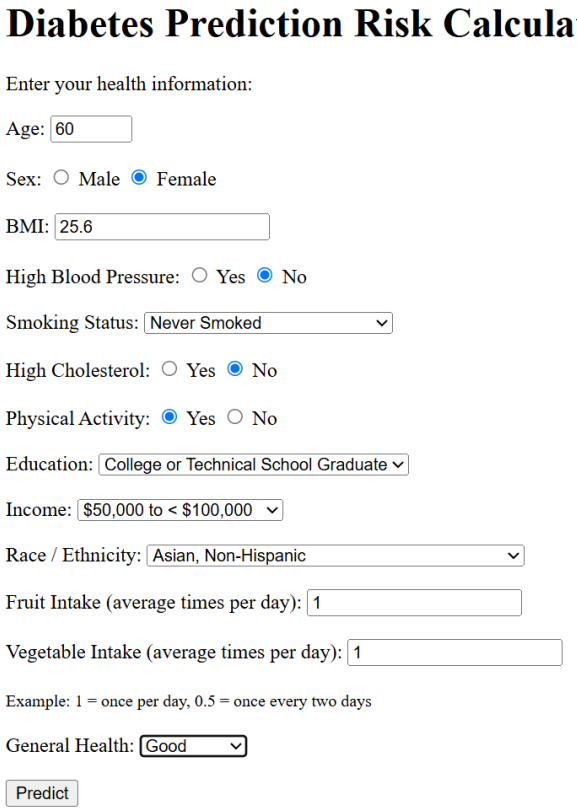


Figure 1. Current 13-field Flask input form used for a hypothetical test case.

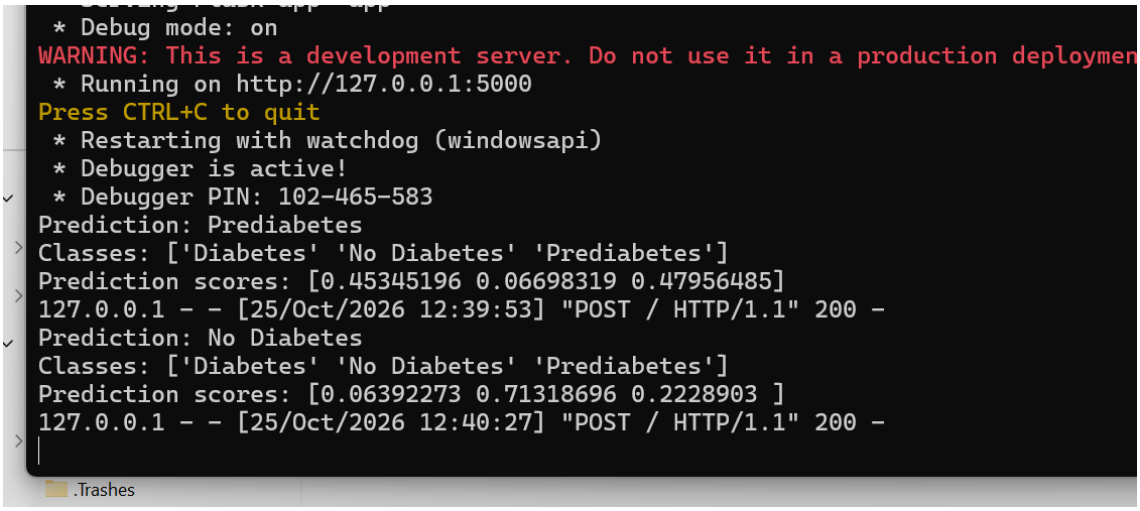


Figure 2. Flask runtime output showing POST requests and model prediction scores during implementation testing.

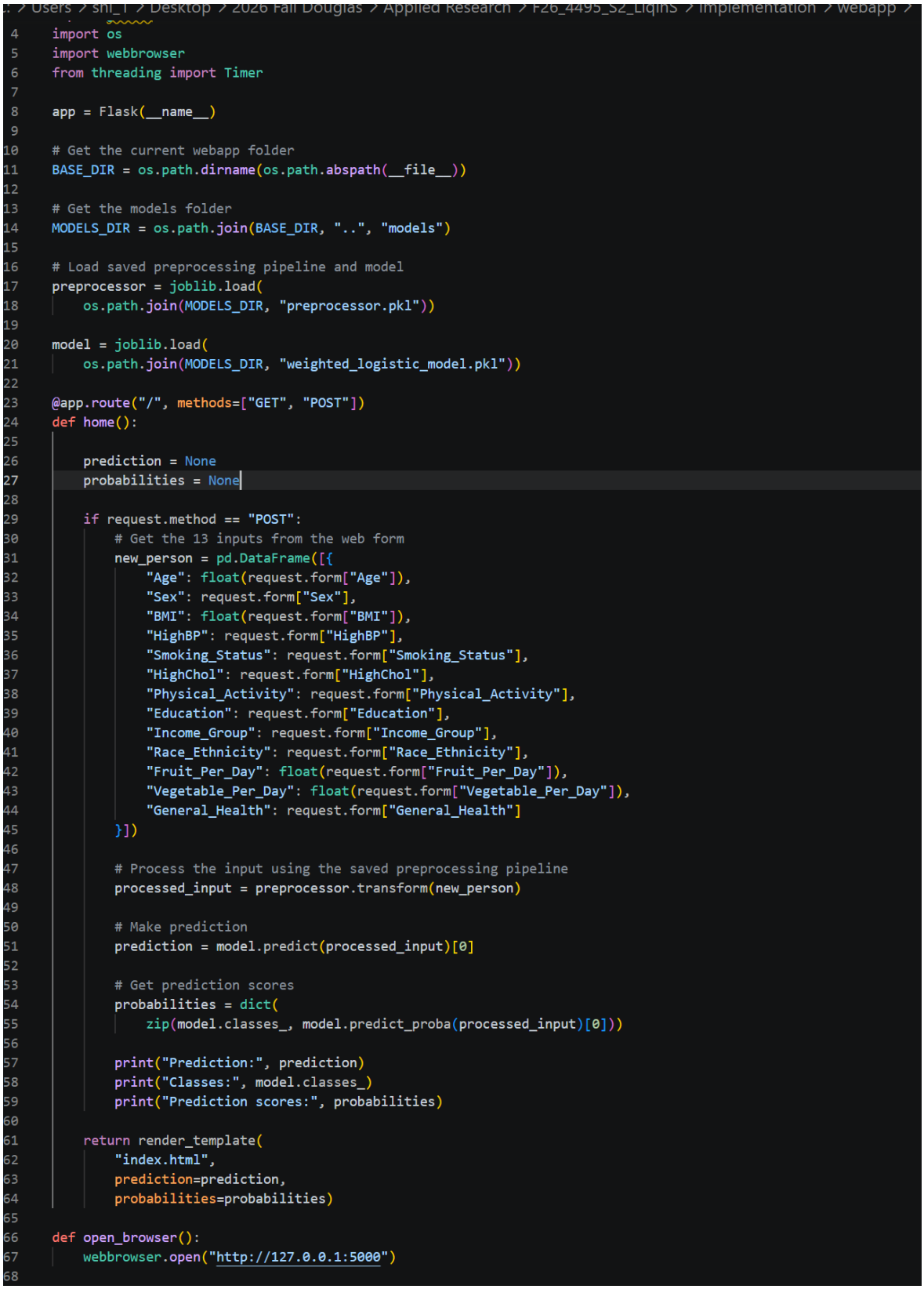
6.2 Design decisions

At MidCourse I intentionally kept the interface simple. Binary fields use radio buttons, categorical variables use drop-down lists, and continuous variables use numeric fields. Fruit and vegetable intake remain numeric because the final model was trained on estimated average times per day, not a yes/no daily-intake variable. The current goal is correct input mapping and prediction. Styling can be improved later without changing the model pipeline.

I also added automatic browser opening when python app.py is run. This is a small usability improvement for the local demonstration. The application is currently a development server only and is intended for a course demo, not public clinical deployment.

6.3 Implementation evidence

The final Flask backend shows the complete prediction path in one place. The route accepts GET and POST requests, reads the 13 form values into a one-row pandas DataFrame, applies the saved preprocessing pipeline, runs predict() and predict_proba(), and passes the predicted class and class scores back to the HTML template. I tested this flow locally before using it for the MidCourse scenario comparisons.



```
import os
import webbrowser
from threading import Timer

app = Flask(__name__)

# Get the current webapp folder
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# Get the models folder
MODELS_DIR = os.path.join(BASE_DIR, "..", "models")

# Load saved preprocessing pipeline and model
preprocessor = joblib.load(
    os.path.join(MODELS_DIR, "preprocessor.pkl"))

model = joblib.load(
    os.path.join(MODELS_DIR, "weighted_logistic_model.pkl"))

@app.route("/", methods=["GET", "POST"])
def home():

    prediction = None
    probabilities = None

    if request.method == "POST":
        # Get the 13 inputs from the web form
        new_person = pd.DataFrame([{
            "Age": float(request.form["Age"]),
            "Sex": request.form["Sex"],
            "BMI": float(request.form["BMI"]),
            "HighBP": request.form["HighBP"],
            "Smoking_Status": request.form["Smoking_Status"],
            "HighChol": request.form["HighChol"],
            "Physical_Activity": request.form["Physical_Activity"],
            "Education": request.form["Education"],
            "Income_Group": request.form["Income_Group"],
            "Race_Ethnicity": request.form["Race_Ethnicity"],
            "Fruit_Per_Day": float(request.form["Fruit_Per_Day"]),
            "Vegetable_Per_Day": float(request.form["Vegetable_Per_Day"]),
            "General_Health": request.form["General_Health"]
        }])

        # Process the input using the saved preprocessing pipeline
        processed_input = preprocessor.transform(new_person)

        # Make prediction
        prediction = model.predict(processed_input)[0]

        # Get prediction scores
        probabilities = dict(
            zip(model.classes_, model.predict_proba(processed_input)[0]))

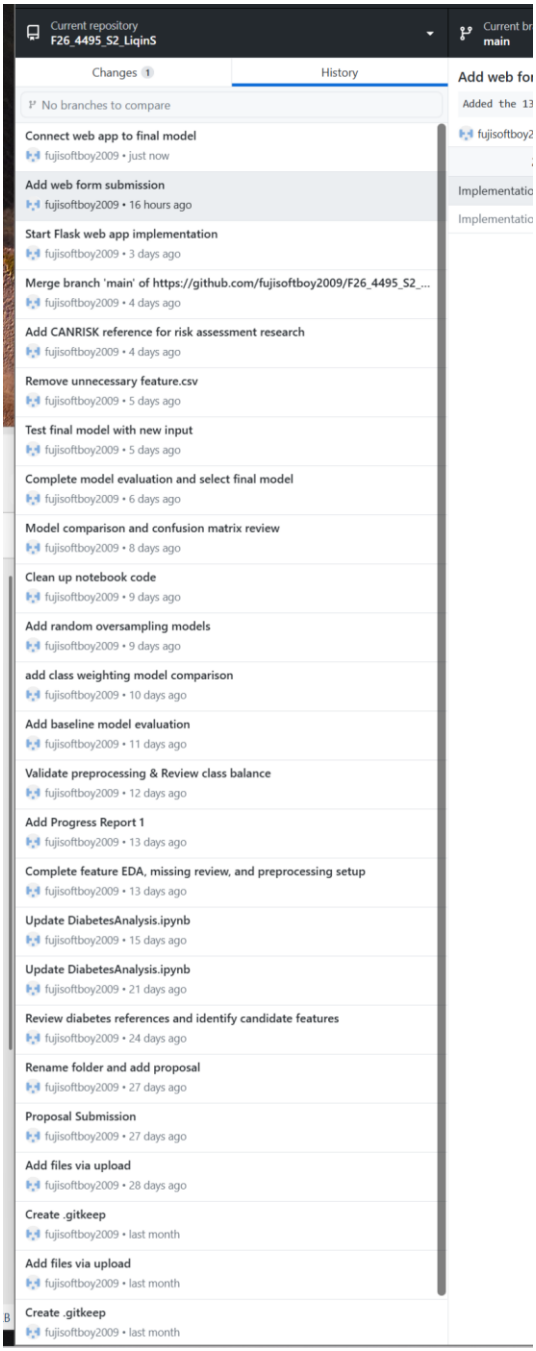
        print("Prediction:", prediction)
        print("Classes:", model.classes_)
        print("Prediction scores:", probabilities)

    return render_template(
        "index.html",
        prediction=prediction,
        probabilities=probabilities)

def open_browser():
    webbrowser.open("http://127.0.0.1:5000")
```

Implementation evidence: final Flask backend in VS Code, including the 13-input DataFrame, preprocessing, prediction, class scores, and return to the web page.

I also used small GitHub commits instead of waiting for one large upload. The history below shows the project moving from preprocessing and model evaluation to model selection and Flask implementation. This provides a clear record of the frequency and consistency of my work and makes each major change easier to review.



Repository evidence: GitHub Desktop history showing regular project commits from data preparation and model development through final model selection and Flask web application implementation.

7. Scenario Comparison with CDC and CANRISK

I used two hypothetical profiles to compare the direction of my model's output with two established public risk-screening tools. Because the inputs and targets are different, this is not formal validation. The purpose is to see whether the overall direction is reasonable and to identify where the tools disagree.

7.1 Case 1: Relatively healthier profile

The shared profile used a 60-year-old female, BMI 25.6, no high blood pressure, physically active, Asian background, and college/university education. For my model I also had to supply variables not asked by the external tools: Never Smoked, No High Cholesterol, income $50,000 to < $100,000, fruit intake 1 time/day, vegetable intake 1 time/day, and General Health = Good. These are explicit assumptions for the comparison.

Tool	Observed result
Canada CANRISK	27 — Moderate
CDC Prediabetes Risk Test	4/10 — Low Risk for Prediabetes
BRFSS ML model	Prediabetes; Prediabetes 52.0%, No Diabetes 30.4%, Diabetes 17.6%

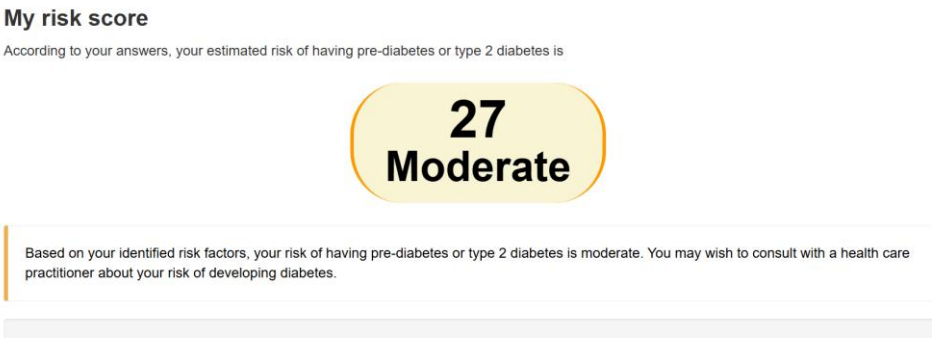


Figure 3. CANRISK result for Case 1: score 27, Moderate.

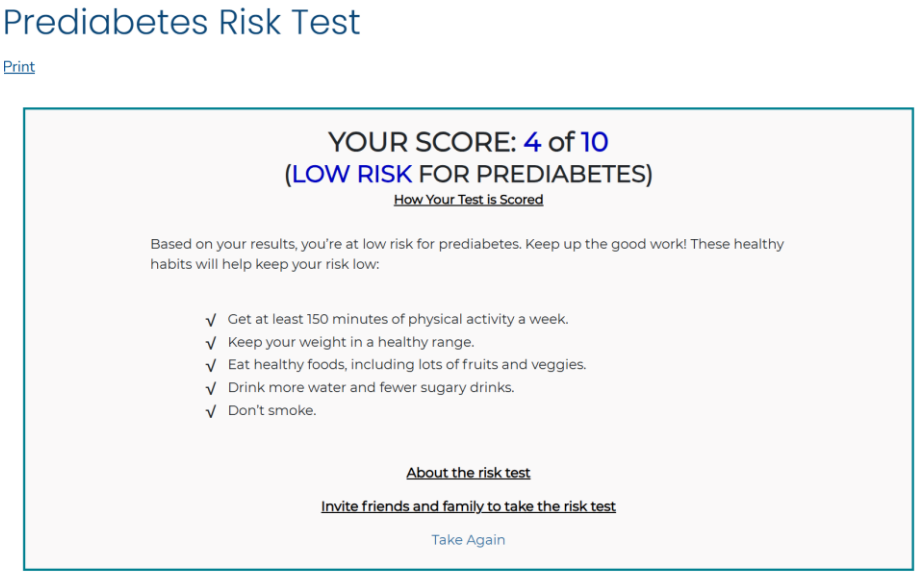


Figure 4. CDC Prediabetes Risk Test result for Case 1: 4/10, Low Risk.

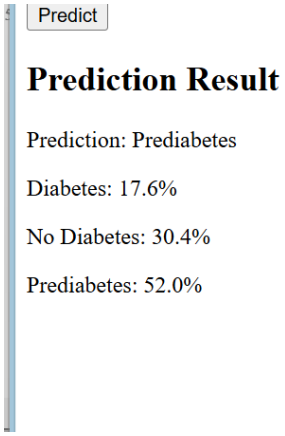


Figure 5. BRFSS model result for Case 1: Prediabetes with a 52.0% model score.

Case 1 shows a difference between the tools. CANRISK gives Moderate risk, CDC gives Low risk, and my model places the highest score on Prediabetes. This difference is not surprising because the scoring rules, predictor sets, and prediction targets are not the same. It also shows why the external tools should not be treated as a direct accuracy test for the BRFSS classifier.

7.2 Case 2: High-risk male profile

The second profile was intentionally higher risk: male, age 65, BMI 34.1, high blood pressure, physically inactive, Asian background, and lower educational level. The external tools also used risk factors that are not present in Model A, including waist circumference, previous high blood sugar, and family history. For my model, additional assumptions were Former Smoker, High Cholesterol, income $25,000 to < $35,000, fruit and vegetable intake 0.5 times/day, and General Health = Fair.

Tool	Observed result
Canada CANRISK	75 — High
CDC Prediabetes Risk Test	9/10 — High Risk for Prediabetes
BRFSS ML model	Diabetes; Diabetes 50.6%, Prediabetes 45.7%, No Diabetes 3.7%

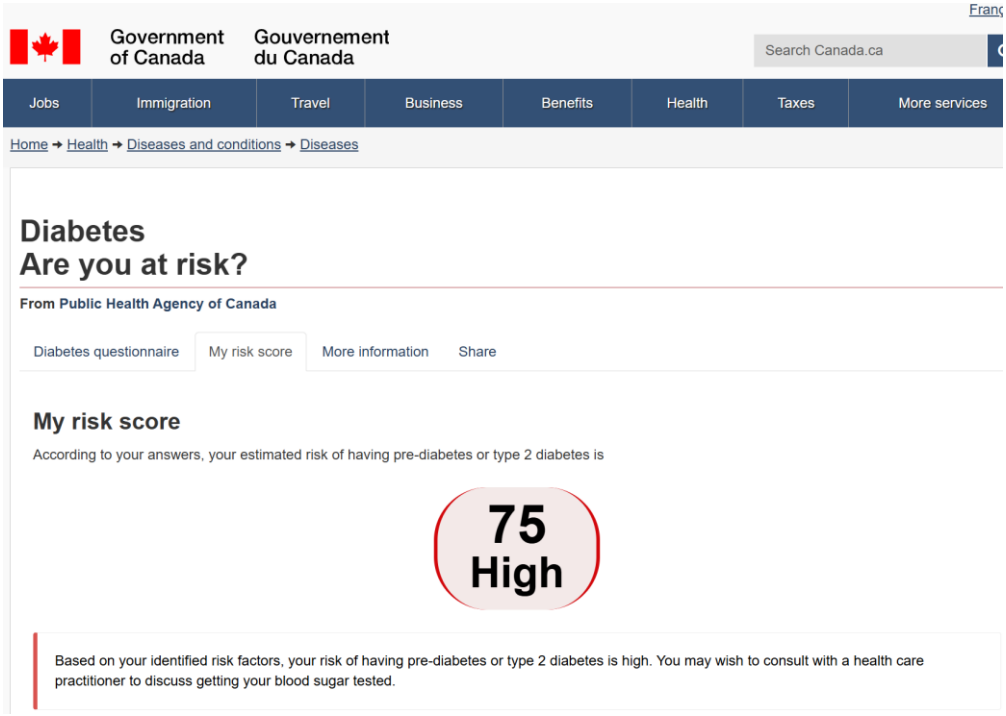


Figure 6. CANRISK result for Case 2: score 75, High.

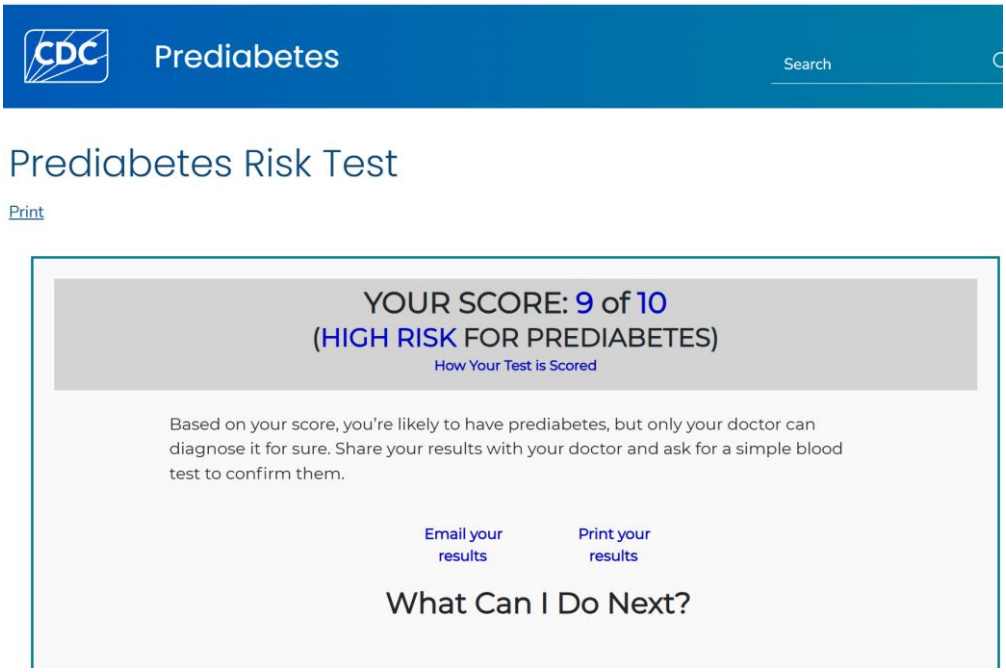


Figure 7. CDC Prediabetes Risk Test result for Case 2: 9/10, High Risk.

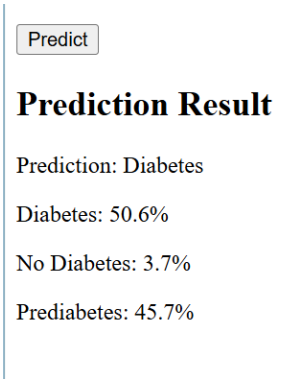


Figure 8. BRFSS model result for Case 2: Diabetes 50.6%, Prediabetes 45.7%, No Diabetes 3.7%.

Case 2 shows strong directional agreement: both public tools report high risk, and my model assigns only 3.7% to No Diabetes while splitting most of the score between Diabetes and Prediabetes. Diabetes (50.6%) and Prediabetes (45.7%) are close, so I would not describe this as a strong or certain Diabetes prediction. The result is better interpreted as the model placing the profile close to the Diabetes/Prediabetes boundary.

7.3 What this comparison does and does not show

The machine-learning model is more granular than a fixed point score in two ways: it uses 13 selected BRFSS predictors and returns three class scores from coefficients learned from the training data. However, this does not prove that it is clinically superior to CDC or CANRISK. CDC and CANRISK were created for risk screening, while my model predicts a self-reported current status label in BRFSS. The two scenario tests are therefore a useful reasonableness check and discussion point, not a validation study.

8. Current Limitations and Next Steps

8.1 Current limitations

- The target is based on self-reported prior diagnosis rather than a uniform laboratory test. Some undiagnosed cases may be labeled No Diabetes, but this cannot be confirmed from BRFSS alone.
- Prediabetes is only 2.88% of the modeling dataset and overlaps with the other classes. Its precision and F1 remain low even after balancing methods.
- The class-weighted Logistic Regression scores are useful model outputs but should not be interpreted as calibrated clinical probabilities.
- The web app currently assumes complete valid input and is still a local development application.
- The two external scenario comparisons use different predictor definitions and cannot establish model accuracy or clinical superiority.
- BRFSS is cross-sectional and self-reported; the project does not predict future diabetes incidence.

8.2 Next steps

- Improve the web interface so that the input form is easier to understand and use. Add short explanations for important fields and useful information about the prediction result.
- Create a small Power BI dashboard to summarize the cleaned BRFSS data and show important patterns related to diabetes and prediabetes.
- Continue reviewing existing diabetes risk tools and research to find additional ways to compare and cross-check the model results. These comparisons will be used as supporting evidence rather than formal clinical validation.
- Decide whether SQLite provides a useful function for the final application. If it does not add meaningful value, document the scope change instead of including an unnecessary database component.
- Complete final testing of the web application, document the project limitations, and prepare the final report and presentation.

9. AI Use

I used ChatGPT as a support tool for explanation, debugging, and writing review. I did not treat AI output as project evidence by itself. I ran the code, checked the CDC documentation, inspected model outputs, made the modeling decisions, tested the application, and revised the report based on my own understanding.

AI tool	Version / account	Specific use	My value addition beyond AI
ChatGPT	GPT-5.6 Sol personal ChatGPT account	Helped discuss BRFSS variables and missing data; explain evaluation metrics and imbalance methods; review model comparisons; troubleshoot Flask/Conda issues; explain joblib model files; refine concise English; compare the purpose of CDC/CANRISK with the BRFSS model.	I selected the dataset and predictors; read the CDC codebook; wrote and ran the notebook; checked special codes and outputs; rejected complete-case deletion; tested the models; inspected confusion matrices; chose Weighted Logistic Regression for the project goal; created and tested the Flask form; mapped the 13 inputs to the model; verified two scenario comparisons; made GitHub commits; and reviewed all report statements before including them.

Response to instructor feedback: This report uses first-person explanations of the decisions I made and the evidence I used. The work log also describes my own actions and reasoning rather than listing only what AI discussed.

10. Work Date / Hours Log

The descriptions below emphasize the work I personally completed and the decisions I made.

Date	Hours	Description of work done
Sep 21, 2026	4	Investigated whether medical notes were suitable for the project, reviewed privacy and scope concerns, and decided not to use those data.
Sep 22, 2026	4	Searched for a replacement public dataset, selected 2025 CDC BRFSS, and reframed the project around diabetes/prediabetes status.
Sep 23, 2026	4	Discussed the CDC data source with the instructor, organized the GitHub repository, and uploaded initial materials.
Sep 24, 2026	2	Studied CDC documentation and planned the data/model/web architecture.
Sep 25, 2026	2	Continued background study and architecture planning.
Sep 27, 2026	5	Reviewed proposal requirements and CDC documentation and prepared the proposal draft.
Oct 1, 2026	4	Reviewed the 2025 BRFSS codebook and calculated-variable documentation; identified candidate variables and compared them with the earlier 2021 diabetes project.
Oct 4, 2026	4	Created the three-class target from DIABETE4, selected _AGE80 for age, created age groups, and reviewed the sex variable.
Oct 5, 2026	2	Reviewed how selected BRFSS variables and calculated variables are defined and how special response codes should be interpreted.
Oct 7, 2026	3	Compared original and calculated BRFSS variables and decided which versions were more suitable for the project.

Date	Hours	Description of work done
Oct 8, 2026	3	Reviewed special response/missing definitions and refined recoding plans for demographic, lifestyle, socioeconomic, and diet variables.
Oct 10, 2026	5	Completed age/sex EDA, calculated diabetes prevalence across age groups, created an age-prevalence chart, and documented observations.
Oct 11, 2026	6	Cleaned and reviewed BMI, blood pressure, smoking, cholesterol, activity, education, income, race/ethnicity, fruit/vegetable intake, and general health using the CDC documentation.
Oct 12, 2026	6	Reviewed missing data, rejected complete-case deletion because it would remove about 40%, built the 13-predictor dataset, split train/test data, set up preprocessing, and prepared Progress Report 1.
Oct 13, 2026	2	Validated the preprocessing pipeline, confirmed 40 processed features and zero post-processing missing values, and reviewed the severe class imbalance before modeling.
Oct 14, 2026	3	Built baseline Logistic Regression and Random Forest models, reviewed classification reports/confusion matrices, and identified Prediabetes failure as the main modeling problem.
Oct 15, 2026	2	Tested class-weighted Logistic Regression and Random Forest, compared class-level recall/F1, and decided to test RandomOverSampler before considering synthetic sampling.
Oct 16, 2026	3	Applied RandomOverSampler to training data only, trained oversampled Logistic Regression/Random Forest, compared six models, and concluded imbalance was not the only Prediabetes problem.
Oct 17, 2026	3	Inspected the Weighted Logistic Regression confusion matrix, compared age/BMI/diet across classes, tested custom weights, and documented class overlap.
Oct 19, 2026	2	Tested HistGradientBoosting and healthcare-access Model B, compared results with Model A, reviewed label limitations, and selected Model A + Weighted Logistic Regression.
Oct 24, 2026	5	Saved the preprocessor and final model with joblib, created the Flask webapp structure, built the 13-field HTML form, tested local loading, and connected form submission to Flask POST.
Oct 25, 2026	5	Connected all 13 form inputs to the saved preprocessing/model pipeline, displayed predicted class and percentages, tested automatic browser opening, ran two CDC/CANRISK comparison scenarios, and prepared the MidCourse report.
Oct 26, 2026	5	Recorded and completed the MidCourse project demonstration video.

11. Closing

At MidCourse, the core technical objective is working: a cleaned 2025 BRFSS dataset, reproducible preprocessing, multiple evaluated classifiers, a selected final model, saved model artifacts, and a Flask interface that accepts the 13 Model A inputs and returns a three-class result. The most important finding so far is that Prediabetes is substantially harder to separate than the other classes. Class weighting improves recall, but it also creates many false positives. I therefore treat this limitation as a result to explain rather than hide.

The next phase is mainly product refinement and communication: Power BI, web interface improvement, additional external comparison, final testing, and final reporting.

12. References

[1] Centers for Disease Control and Prevention. 2025 BRFSS Survey Data and Documentation. https://www.cdc.gov/brfss/annual_data/annual_2025.html

[2] Centers for Disease Control and Prevention. 2025 BRFSS Codebook (available from the 2025 BRFSS data page). https://www.cdc.gov/brfss/annual_data/annual_2025.html

[3] Centers for Disease Control and Prevention. About the Prediabetes Risk Test. https://www.cdc.gov/prediabetes/risktest/about-the-test.html

[4] Centers for Disease Control and Prevention. How Your Prediabetes Risk Test Is Scored. https://www.cdc.gov/diabetes/widgets/risktest/how-your-test-is-scored.html

[5] Public Health Agency of Canada. The Canadian Diabetes Risk Questionnaire (CANRISK). https://health.canada.ca/apps/canrisk-standalone/pdf/canrisk-en.pdf

[6] Public Health Agency of Canada. Diabetes: Overview. Government of Canada. https://www.canada.ca/en/public-health/services/diseases/diabetes.html

[7] Diabetes Canada. Diabetes in Canada. https://www.diabetes.ca/what-is-diabetes/diabetes-in-canada

Appendix A. AI Prompt History

The following screenshots show selected ChatGPT prompts used during the project for model evaluation, feature decisions, imbalance handling, and Flask implementation.

Prompt 1: Handling Prediabetes class imbalance

Prediabetes only take 2.9% of the traning data, should I build the modles without balancing? and then compare methods such as class weighting or SMOTE? Do you have any recommendation?

Yes. I would recommend building the **baseline models first without balancing**, and then comparing balancing methods.

A good sequence would be:

1. Train Logistic Regression and Random Forest on the original training data.
2. Evaluate each model using the confusion matrix, precision, recall, and F1-score for each class.
3. Pay special attention to **Prediabetes recall and F1-score**, because this class is only about 2.9%.
4. Then try **class weighting** and compare the results.
5. If needed, test **SMOTE** on the training data only.

I would not use SMOTE immediately, because you first need a baseline to see how serious the imbalance problem is. Class weighting is also simpler and usually a good first method to test.

The final model should not be selected by accuracy alone. If one model has slightly lower accuracy but much better Prediabetes recall and F1-score, it may be more useful for your project.

so basically step by step

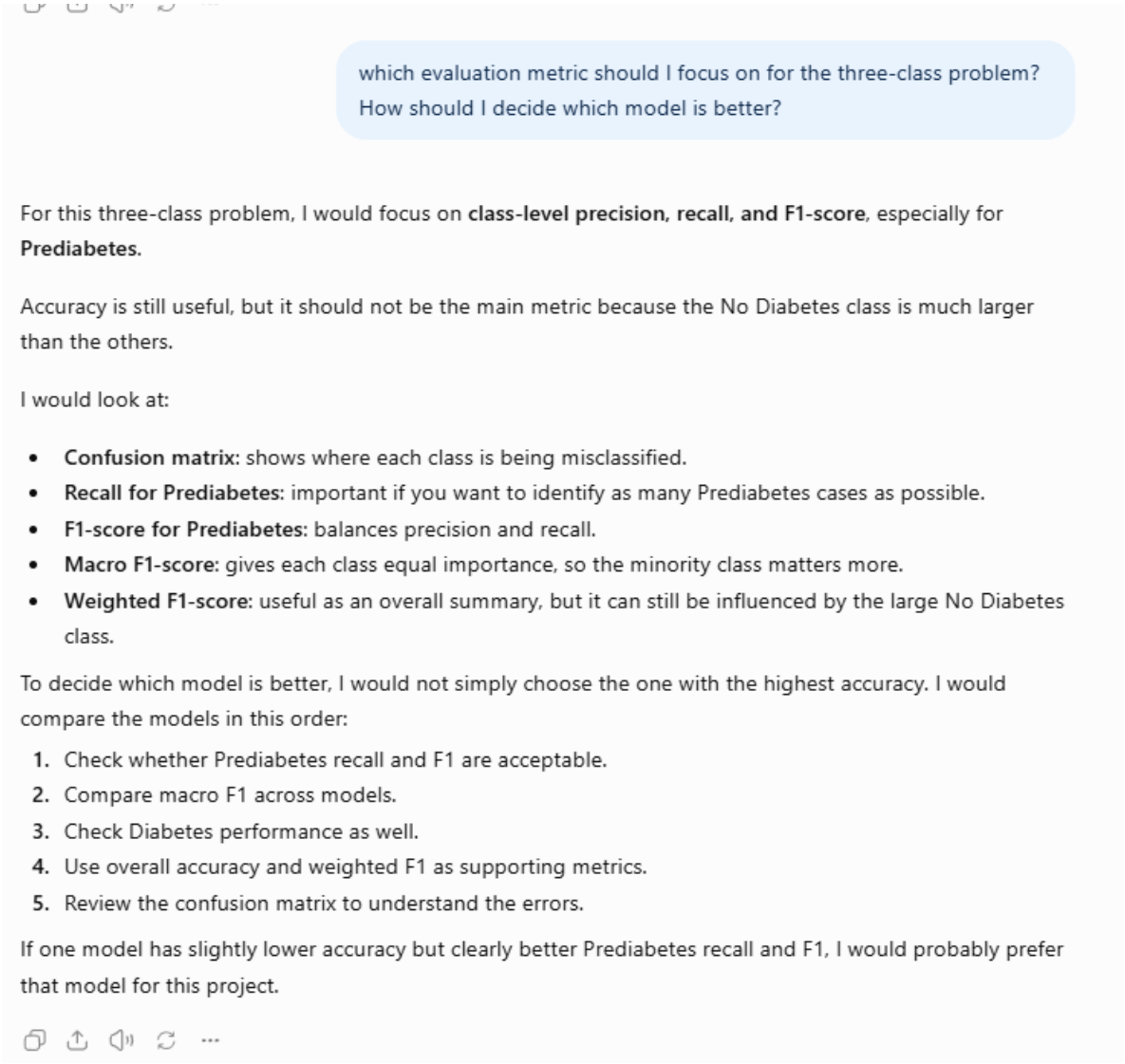
Exactly. Step by step is the better approach here.

First, build the baseline models on the original imbalanced training data. Then evaluate how well they identify each class, especially Prediabetes. After that, try class weighting. If the Prediabetes results are still weak, test SMOTE on the training data only.

That way, you can clearly show whether each imbalance method actually improves the model instead of applying one automatically from the beginning.

Appendix Figure A1. Prompt 1: Handling Prediabetes class imbalance.

Prompt 2: Evaluation metrics for the three-class problem



Appendix Figure A2. Prompt 2: Evaluation metrics for the three-class problem.

Prompt 3: Baseline model comparison

Logistic Regression has slightly higher accuracy, but Random Forest has better Diabetes F1. Which model should I consider better at this stage?

At this stage, I would not choose a final model yet.

Logistic Regression has slightly higher accuracy, but Random Forest performs a little better for Diabetes and has a slightly higher macro F1-score. However, both models perform very poorly for Prediabetes.

Because Prediabetes is important in this project, I would keep both as baseline models and test class imbalance methods next. After that, compare the models again using Prediabetes recall/F1 and macro F1, not accuracy alone.

Appendix Figure A3. Prompt 3: Baseline model comparison.

Prompt 4: Class weighting before SMOTE

Yes. I would try **class weighting first**, then use SMOTE only if the improvement is still not enough.

Class weighting is simpler because it keeps the original training data and only tells the model to give more importance to the minority classes. This makes it a good first comparison with your baseline models.

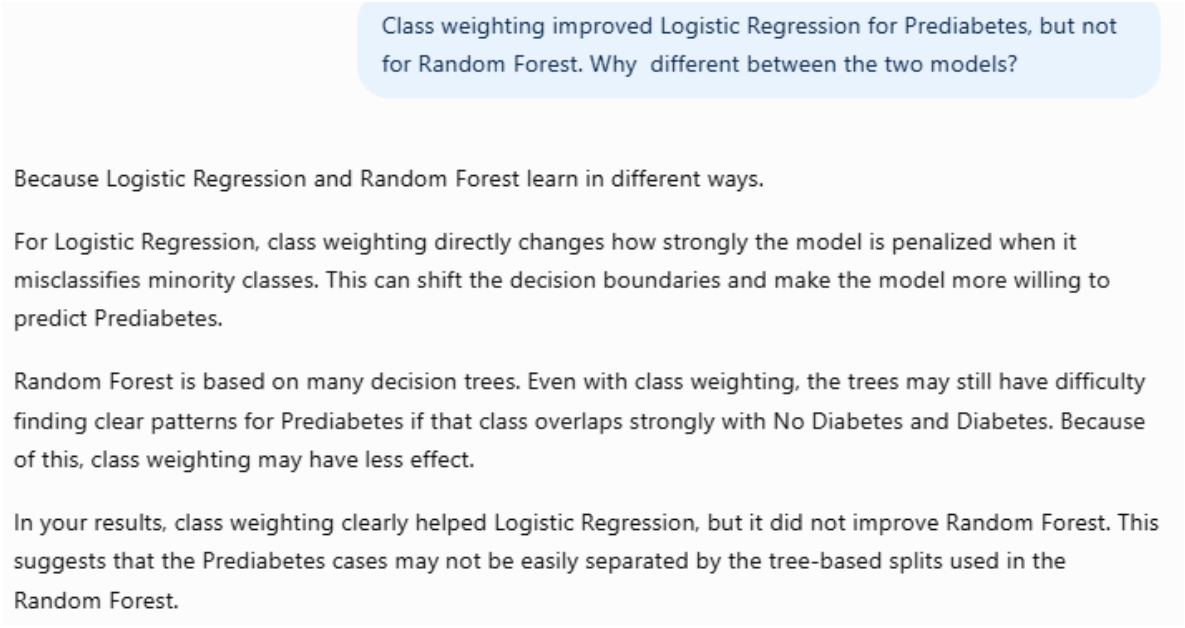
A reasonable order would be:

1. Baseline Logistic Regression and Random Forest
2. Logistic Regression with `class_weight='balanced'`
3. Random Forest with `class_weight='balanced'`
4. Compare Prediabetes recall, F1-score, and macro F1
5. If Prediabetes performance is still very poor, then try SMOTE on the training data only

I would not jump directly to SMOTE because it creates synthetic samples. It is better to first see whether a simpler method like class weighting is enough.

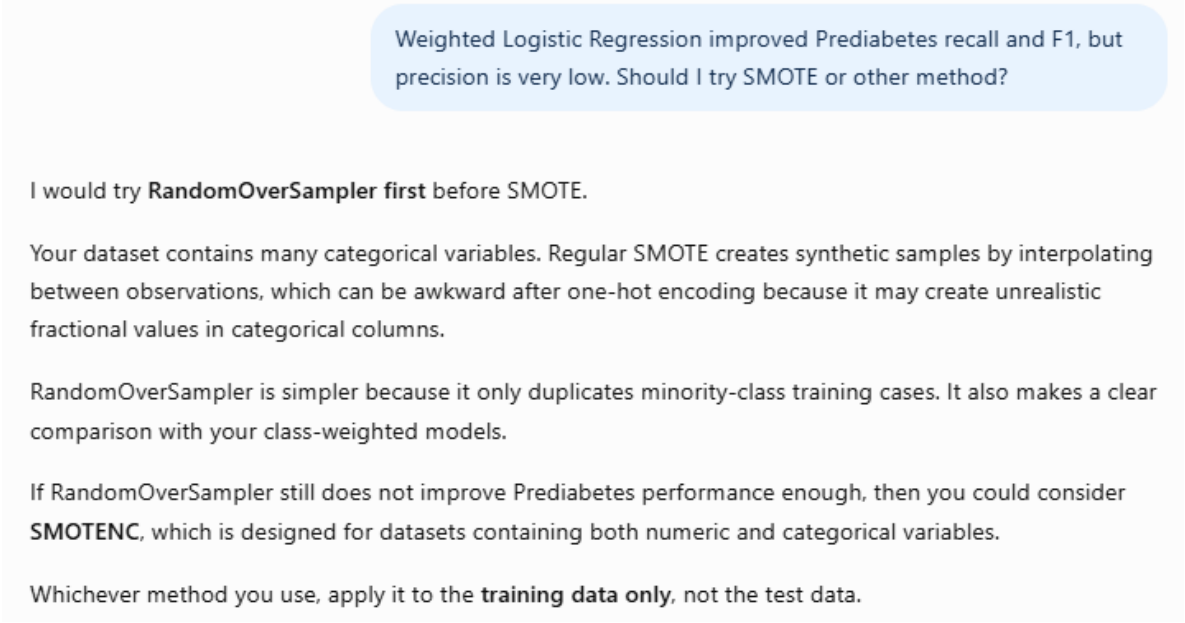
Appendix Figure A4. Prompt 4: Class weighting before SMOTE.

Prompt 5: Why class weighting affected Logistic Regression and Random Forest differently



Appendix Figure A5. Prompt 5: Why class weighting affected Logistic Regression and Random Forest differently.

Prompt 6: RandomOverSampler as the next balancing method



Appendix Figure A6. Prompt 6: RandomOverSampler as the next balancing method.

Prompt 7: Why Prediabetes remained difficult after oversampling

The results showed Random oversampling improved Random Forest a little bit, but the Prediabetes still very poor. Is it becuase the class imbalance?

Yes, **class imbalance is part of the reason**, but probably not the only reason.

Prediabetes is only about 2.9% of the original training data, so the model has far fewer examples to learn from. Random oversampling gives the model more chances to learn from those cases, which explains the small improvement.

However, Prediabetes performance is still very poor even after balancing the classes. This suggests that Prediabetes may also **overlap with both No Diabetes and Diabetes** in the predictor variables. In other words, the features may not separate Prediabetes clearly.

So at this stage, I would say the problem is likely caused by **both class imbalance and difficulty separating the Prediabetes class**.

what is your suggestion to solve the problem?

I would not keep increasing oversampling, because your results suggest that imbalance is only part of the problem.

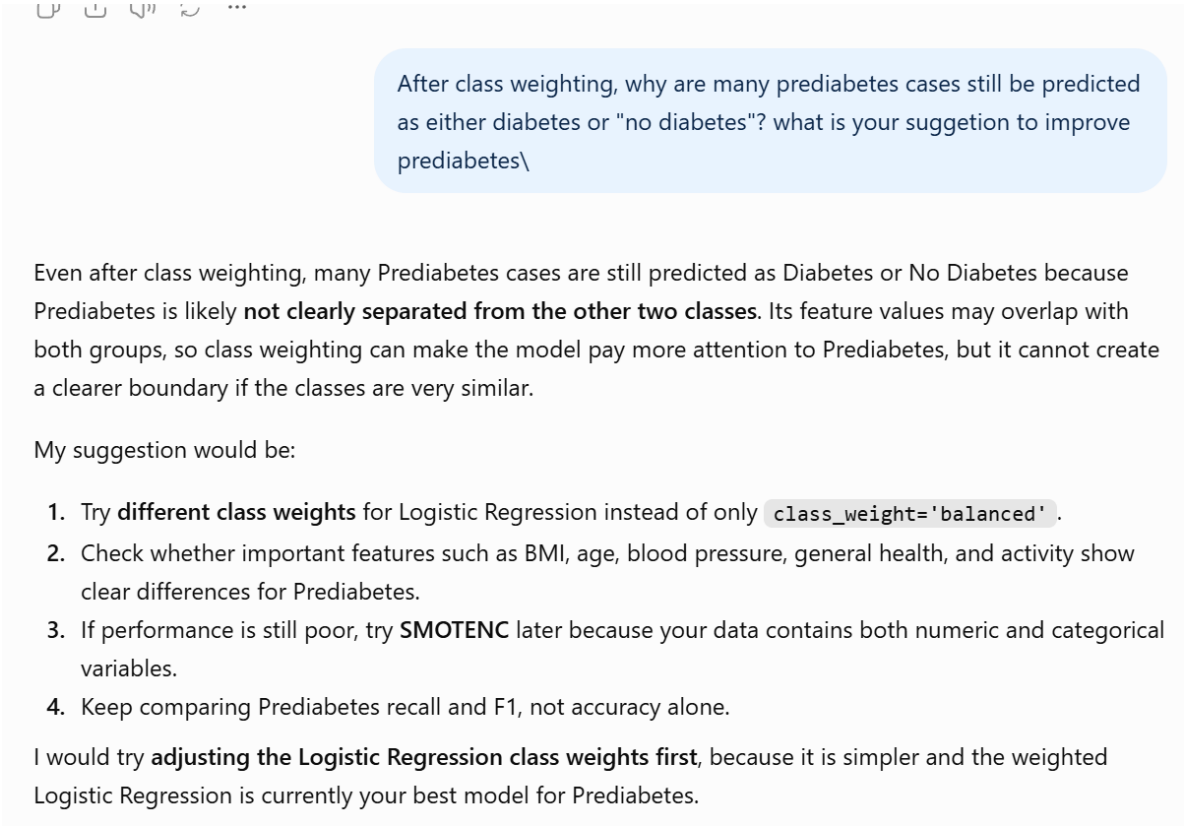
My next steps would be:

1. **Tune the class weights for Logistic Regression** instead of using only `class_weight='balanced'`. The balanced version increased Prediabetes recall, but precision became very low, so a less aggressive weight may give a better balance.
2. If that is still not enough, try **SMOTENC**, because your dataset contains both numerical and categorical variables.
3. Compare the results using **Prediabetes precision, recall, F1, and macro F1**, not accuracy alone.
4. If Prediabetes is still difficult to predict, examine whether its feature values overlap heavily with No Diabetes and Diabetes.

If the performance remains poor after these steps, that would also be an important project finding: the available BRFSS predictors may not clearly separate Prediabetes from the other two classes.

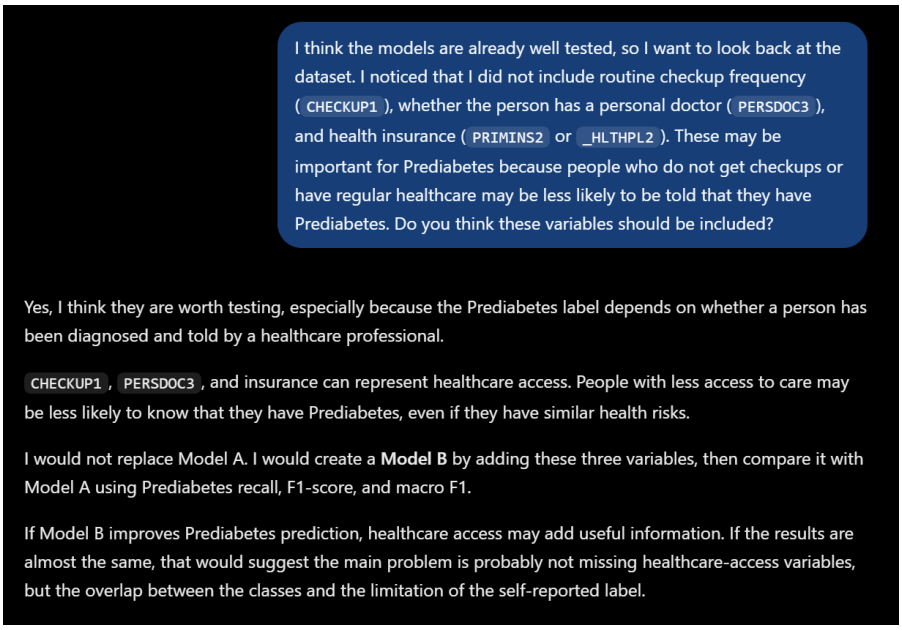
Appendix Figure A7. Prompt 7: Why Prediabetes remained difficult after oversampling.

Prompt 8: Why many Prediabetes cases were still misclassified after class weighting



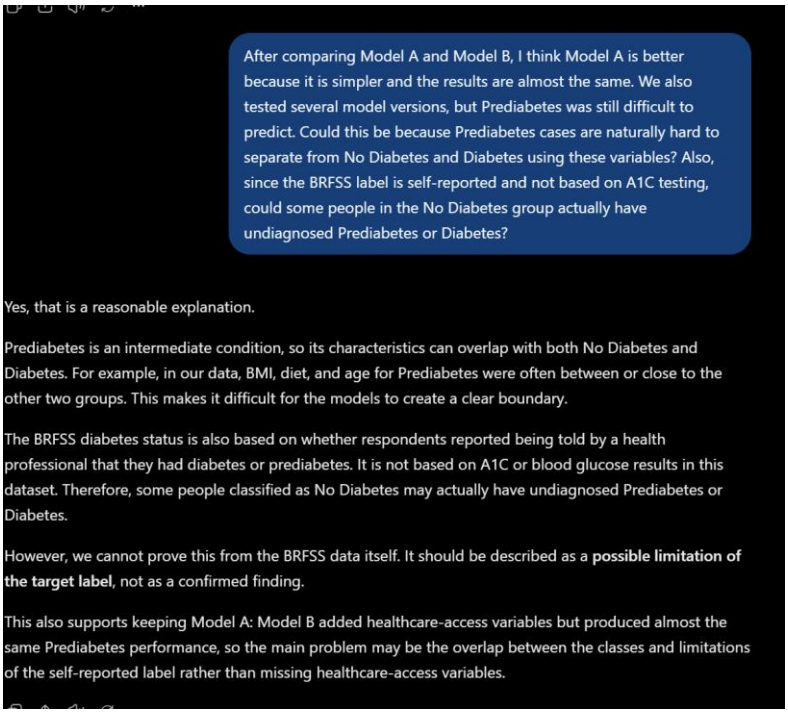
Appendix Figure A8. Prompt 8: Why many Prediabetes cases were still misclassified after class weighting.

Prompt 9: Whether healthcare-access variables should be added



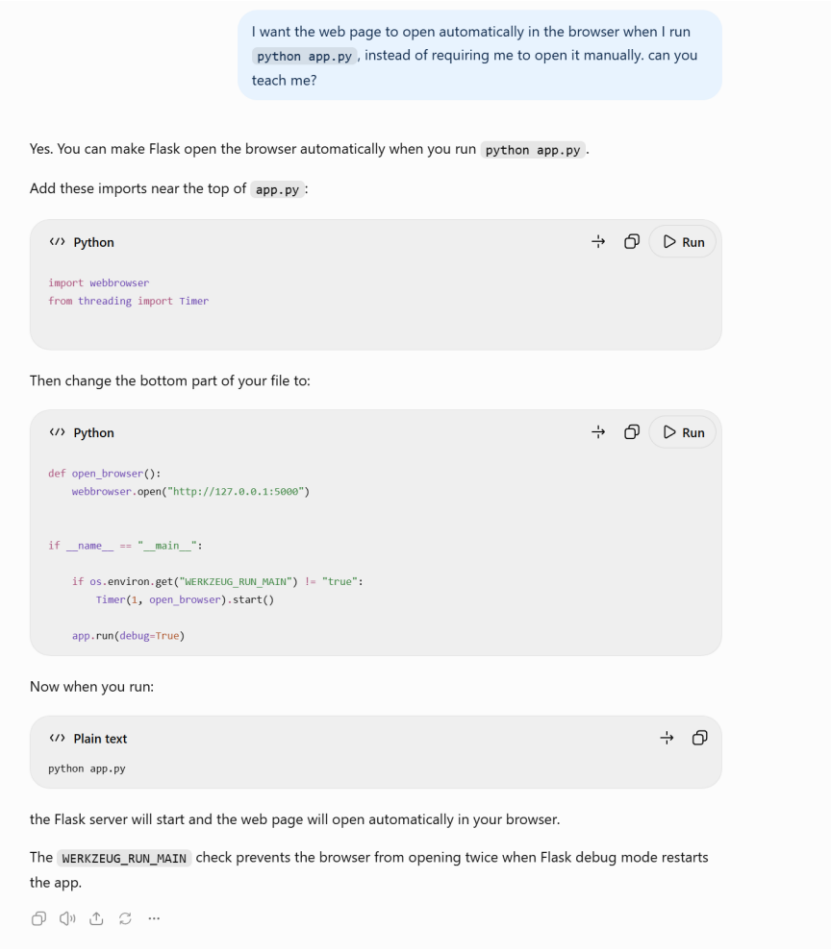
Appendix Figure A9. Prompt 9: Whether healthcare-access variables should be added.

Prompt 10: Model A vs Model B and self-reported label limitations



Appendix Figure A10. Prompt 10: Model A vs Model B and self-reported label limitations.

Prompt 11: Automatically opening the browser when the Flask app starts



Appendix Figure A11. Prompt 11: Automatically opening the browser when the Flask app starts.